

SYMULATOR NOZOWNIKA

Dokumentacja techniczna projektu

Autorzy

Piotr Poletylo

Michał Paczkowski

Dawid Malecki

ASP.NET Core 10 MVC | Entity Framework Core 10 | SQL Server | C#

Czerwiec 2026

1. Opis projektu

Symulator Nożownika to wieloużytkownikowa aplikacja webowa zrealizowana w technologii ASP.NET Core 10 MVC. Projekt łączy elementy gry przeglądarkowej z rozbudowanym systemem zarządzania profilem gracza, mechanikami RPG oraz narzędziem społecznościowym. Użytkownicy rejestrują konta, zdobywają doświadczenie, awansują na kolejne poziomy, wykonują questy, nabywają bron i eliksiry w sklepie oraz rywalizują w globalnych rankingach.

Repozytorium: https://github.com/Piiootreek/Symulator_Nożownika

2. Architektura i stack technologiczny

Aplikacja opiera się na wzorcu MVC wbudowanym w platformę ASP.NET Core. Poniżej zestawienie głównych komponentów:

Warstwa	Technologia
Backend	ASP.NET Core 10 MVC, C#
ORM / baza danych	Entity Framework Core 10, Microsoft.EntityFrameworkCore.SqlServer 10.0.5

Autoryzacja	Cookie Authentication
Frontend	Razor Views (.cshtml), Bootstrap 5,
REST API (wewnętrzne)	ReportsApiController, JSON, autoryzacja przez nagłówki HTTP
API zewnętrzne	BrowserService / IBrowserService, PuppeteerSharp 14.0.0
Generowanie PDF	ReportService / IReportService
IDE / VCS	Visual Studio, Git (GitHub)

3. Wymagania niefunkcjonalne -- spełnienie i uzasadnienie

WN-1 Obsługa formularzy (wyświetlanie, usuwanie, dodawanie, edycja)

Spełnione.

Aplikacja zawiera formularze Razor we wszystkich kluczowych obszarach: rejestracja konta (RegistrationViewModel), logowanie (LoginViewModel), zmiana hasła (ChangePasswordViewModel), tworzenie i edycja wpisów w klubie, zarządzanie questami oraz formularz zgłoszeń. Każdy formularz implementuje walidacje po stronie serwera z użyciem atrybutów Data Annotations i zwraca komunikaty błędów do widoku przez ModelState. Operacje CRUD realizowane są przez akcje: Index (lista), Details, Create (GET+POST), Edit (GET+POST), Delete (GET+POST).

WN-2 Moduł logowania i autoryzacja użytkownika

Spełnione.

AccountController implementuje pełny cykl autoryzacji: rejestrację nowego użytkownika z haszowaniem hasła, logowanie z weryfikacją danych, wylogowanie oraz zmianę hasła. Sesja zarządzana jest przez Cookie Authentication (SignInAsync / SignOutAsync). Kontrolery chronione są atrybutem [Authorize], natomiast [DemoBlockAttribute] blokuje dostęp do wybranych akcji w trybie demonstracyjnym. Klasa UserAccount przechowuje dane uwierzytelniające wraz z rolą użytkownika.

WN-3 Odwołania do API zewnętrznych

Spełnione.

Program pobiera listę państw z osobnego API, aby nie musieć wypisywać ich wszystkich podczas rejestracji

WN-4 Udostępnianie interfejsu REST API

Spełnione.

Kontroler GameScoreController zarządza wynikami graczy, rankingami i statystykami.

API dostępne pod adresem bazowym /api/ReportsApi pozwala na zarządzanie zgłoszeniami użytkowników i wiadomości oraz nakładanie i obsługę kar.

Szczegóły opisane w osobnej dokumentacji

WN-5 Baza danych i trwale przechowywanie informacji

Spełnione.

AppDbContext (Entity Framework Core 10, pakiet Microsoft.EntityFrameworkCore.SqlServer 10.0.5) definiuje DbSet dla wszystkich encji. Migracje tworzą pełny schemat bazy danych. EF Core zapewnia trwałość danych pomiędzy sesjami, oraz integralność. Relacje między encjami modelowane są przez klucze obce i właściwości nawigacyjne.

WN-6 Liczność encji -- minimum 10 na osobę (30 ogółem dla 3 osób)

Spełnione.

Folder Models/ zawiera 32 pliki, folder Data/ zawiera 2 pliki (UserAccount, AppDbContext). Łącznie projekt liczy 34 klasy modelu. Poniżej pełny wykaz:

Lp.	Klasa	Folder	Opis
1	UserAccount	Data/	Konto użytkownika: dane logowania, rola, status konta
2	AppDbContext	Data/	Kontekst bazy danych EF Core, rejestracja wszystkich DbSet
3	Achievement	Models/	Definicja osiągnięcia (nazwa, opis, warunek odblokowania)
4	AchievementType	Models/	Kategoria / typ osiągnięcia
5	ChangePasswordViewModel	Models/	Dane formularza zmiany hasła
6	Club	Models/	Klub gracza: nazwa, opis, właściciel
7	ClubMember	Models/	Relacja gracz-klub (członkostwo)
8	ClubMessage	Models/	Wiadomość na tablicy klubu
9	ClubViewModels	Models/	ViewModels dla widoków klubu (lista, szczegóły)
10	CoinWallet	Models/	Portfel monet gracza (saldo)
11	DemoSession	Models/	Sesja trybu demonstracyjnego
12	ErrorViewModel	Models/	Model błędu aplikacji (strona 404/500)
13	FavoriteWeapon	Models/	Aktywna / ulubiona broń gracza
14	HighScore	Models/	Najlepszy wynik gracza
15	Level	Models/	Definicja poziomu: prog XP, nagrody za awans
16	LoginViewModel	Models/	Dane formularza logowania
17	MessageReport	Models/	Zgłoszenie wiadomości klubowej do moderacji
18	Potion	Models/	Definicja eliksiru: typ, efekt, cena
19	PublicProfileViewModel	Models/	Dane publicznego profilu gracza

20	PurchasedPotion	Models/	Eliksiry zakupione przez gracza
21	PurchasedWeapon	Models/	Bron zakupiona przez gracza
22	PurchasedWeaponUpgrade	Models/	Zakupione ulepszenia do konkretnej broni
23	Quest	Models/	Definicja questa: cel, nagroda, typ
24	QuestType	Models/	Typ questa (dzienny, tygodniowy, jednorazowy)
25	RegistrationViewModel	Models/	Dane formularza rejestracji
26	SavedScore	Models/	Zapis wyniku z pojedynczej sesji gry
27	ShopViewModel	Models/	Dane widoku sklepu (lista broni, eliksirów, saldo)
28	UserAchievement	Models/	Odblokowane osiągnięcia danego gracza
29	UserPenalty	Models/	Kara nałożona na gracza (ban, ostrzeżenie, data wygaśnięcia)
30	UserQuestProgress	Models/	Postęp gracza w realizacji questa
31	UserReport	Models/	Zgłoszenie gracza przez innego użytkownika
32	UserStatistics	Models/	Statystyki gracza: poziom, XP, wygrane, przegrane
33	Weapon	Models/	Definicja broni: nazwa, obrażenia, cena
34	WeaponUpgrade	Models/	Dostępne ulepszenia broni

Łącznie klas modelu: 34. Wymaganie minimalne dla 3 osób: 30. Warunek spełniony z nadwyżką.

WN-7 Przejrzystość i estetyka interfejsu

Spełnione.

Za pomocą CSS i Bootstrapa stworzono interfejs graficzny, który jest przejrzysty i responsywny.

WN-8 Obsługa załączników (formularze i pobieranie plików)

Spełnione.

Formularze obsługujące upload plików korzystają z IFormFile (enctype multipart/form-data). Przesłane pliki przechowywane są na serwerze w folderach wwwroot/avatars i wwwroot/images/potions. Osobne endpointy Download w ReportController umożliwiają pobieranie wygenerowanych raportów PDF jako FileResult z Content-Type application/pdf.

WN-9 Generowanie dokumentów (PDF)

Spełnione.

Projekt realizuje generowanie raportów w formacie PDF. ReportController oraz ReportsApiController wywołują IReportService (implementacja ReportService), który buduje

dokument PDF z danymi statystycznymi gracza. Wygenerowany plik zwracany jest jako FileResult i można go pobrać bezpośrednio z przeglądarki.

4. Wymagania funkcjonalne

4.1 Moduł autoryzacji i zarządzania kontem

Kontrolery / klasy: AccountController, UserAccount, LoginViewModel, RegistrationViewModel, ChangePasswordViewModel, UserPenalty.

- Rejestracja nowego konta z walidacją danych (nazwa użytkownika, hasło, e-mail)
- Logowanie z weryfikacją hasła i komunikatami o błędnych danych
- Wylogowanie i unieważnienie sesji cookie
- Zmiana hasła przez zalogowanego użytkownika
- Publiczny profil gracza (PublicProfileViewModel) -- podgląd statystyk i osiągnięć
- Usuwanie konta
- Blokada konta przez administratora (UserPenalty -- ban z datą wygasnięcia)
- Automatyczne wygasanie kar tymczasowych (PenaltyExpirationBackgroundService)

4.2 Formularze CRUD

Pełne operacje Create, Read, Update, Delete dostępne dla encji: Weapon, Potion, Quest, Club, ClubMessage, Achievement, UserReport. Każdy formularz obsługuje walidację po stronie serwera (ModelState).

4.3 System sklepowy i ekwipunek

Kontrolery / klasy: ShopViewModel, CoinWallet, PurchasedWeapon, PurchasedPotion, FavoriteWeapon, WeaponUpgrade.

- Przeglądanie dostępnych broni i eliksirów z cenami
- Zakup przedmiotów za monety z walidacją salda portfela (CoinWallet)
- Ekwipunek gracza -- lista zakupionych broni i eliksirów
- Ulepszanie broni (WeaponUpgrade, PurchasedWeaponUpgrade)
- Wybór aktywnej broni (FavoriteWeapon)

4.4 Rozgrywka i mechaniki gry

Kontrolery / klasy: GameScoreController, SavedScore, HighScore, UserStatistics, Level, LevelService.

- Sesja gry przeglądarkowej z zapisem wyniku do bazy (SavedScore)

- System doświadczenia (XP) i poziomów -- automatyczny awans po przekroczeniu progu XP
- Nagrody za wyniki: monety, XP, odblokowanie osiągnięć
- Historia zapisanych wyników i najlepsze wyniki (HighScore)
- Statystyki walki: wygrane, przegrane, średni wynik (UserStatistics)

4.5 Klub i komunikacja

Kontrolery / klasy: ClubController, Club, ClubMember, ClubMessage, ClubViewModels.

- Tworzenie i zarządzanie klubem -- właściciel może edytować dane i usuwać wiadomości
- Dołączanie i opuszczanie klubu przez gracza
- Tablica wiadomości klubu -- dodawanie i usuwanie postów
- Lista członków klubu z ich rolami
- Zgłaszanie wiadomości do moderacji (MessageReport)

4.6 System questów i osiągnięć

Kontrolery / serwisy: QuestController, QuestService (IQuestService), AchievementService (IAchievementService).

- Przeglądanie dostępnych questów z opisem celu i nagrody
- Przyjmowanie i śledzenie postępu questów (UserQuestProgress)
- Automatyczne zaliczenie questu po spełnieniu warunku przez QuestService
- System osiągnięć odblokowywanych za działania w grze (AchievementService, UserAchievement)

4.7 Statystyki i rankingi

Kontrolery / klasy: StatisticsController, UserStatistics, HighScore.

- Globalna tablica wyników -- ranking graczy według najlepszego wyniku, XP lub poziomu
- Osobiste statystyki gracza: poziom, XP, liczba walk, najlepszy wynik
- Dane z zewnętrznych API prezentowane w widoku statystyk (BrowserService)

4.8 System raportowania i moderacji

Kontrolery / serwisy: ReportController, ReportsApiController, ReportService, ReportManagementService, IReportService.

- Zgłaszanie graczy za nieodpowiednie zachowanie (UserReport z opisem)
- Zgłaszanie wiadomości klubowych (MessageReport)
- Panel administratora -- lista oczekujących zgłoszeń

- Nakładanie kar: ostrzeżenie, ban tymczasowy, ban permanentny (UserPenalty)
- Automatyczne wygasanie banów tymczasowych (PenaltyExpirationBackgroundService)
- Historia nakładanych kar widoczna dla administratora

4.9 Tryb Demo

Kontrolery / klasy: DemoController, DemoService, DemoSession, DemoBlockAttribute.

- Możliwość uruchomienia aplikacji bez rejestracji konta
- Ograniczony dostęp -- akcje oznaczone [DemoBlockAttribute] są zablokowane
- DemoSession przechowuje stan sesji demonstracyjnej w bazie

4.10 Generowanie raportów PDF

Kontrolery / serwisy: ReportController, ReportsApiController, IReportService, ReportService.

- Generowanie raportu statystyk gracza w formacie PDF
- Pobieranie wygenerowanego pliku z przeglądarki (FileResult, Content-Type: application/pdf)
- Dostęp do generowania raportu również przez REST API

4.11 REST API aplikacji i integracja z zewnętrznymi API

Kontrolery / serwisy: ReportsApiController, BrowserService, IBrowserService.

- REST API: endpointy GET/POST dla danych raportów, odpowiedzi w formacie JSON
- Autoryzacja dostępu do API przez nagłówki HTTP
- Dokumentacja API w Docs/REPORTS.md

4.12 Obsługa załączników i pobieranie plików

- Upload plików przez formularze HTML (IFormFile, enctype multipart/form-data)
- Przechowywanie przesłanych plików na serwerze (wwwroot/avatars, wwwroot/images/potions)
- Pobieranie wygenerowanych raportów PDF przez akcje Download
- Walidacja rozszerzenia i rozmiaru przesyłanego pliku

5. Model danych -- główne relacje

- UserAccount (1) -- (1) UserStatistics

- UserAccount (1) -- (1) CoinWallet
- UserAccount (1) -- (N) PurchasedWeapon
- UserAccount (1) -- (N) PurchasedPotion
- UserAccount (1) -- (1) FavoriteWeapon
- UserAccount (1) -- (N) UserQuestProgress
- UserAccount (1) -- (N) UserAchievement
- UserAccount (1) -- (N) SavedScore
- UserAccount (1) -- (N) UserPenalty
- UserAccount (1) -- (N) UserReport
- UserAccount (1) -- (N) DemoSession
- Club (1) -- (N) ClubMember
- Club (1) -- (N) ClubMessage
- Weapon (1) -- (N) WeaponUpgrade
- PurchasedWeapon (1) -- (N) PurchasedWeaponUpgrade
- Quest (N) -- (1) QuestType
- Achievement (N) -- (1) AchievementType